Department Of Computer Science

Project Proposal Summer 26-27

Course Title	00931 Advanced Programming in Web Technology	Section	E	Group	1

- Institution and department : AIUB, BSc in CSE.
- Submission date : 17/10/26

- **Team members: (The Beekeeper)**

Name	Student ID
Sagar Biswas	22-47929-2
Md. Minhazul Hassan	23-53049-3

- **Course Teacher** : Sultanul Arifeen Hamim
- **Project Title** : MultiHAT Academy

➢ Purpose/Project Goal

A full-stack micro-credential & e-commerce platform that transforms technical notebooks into premium, verifiable digital learning products.

➢ Project Overview

MultiHAT Academy sells premium e-books with buyer-specific dynamic watermarks, offers paid web chapters and interactive quizzes, issues verifiable certificates of completion, and features a User Wallet ecosystem with Referral and Certification Showcase Rewards for organic growth. As both a revenue-generating platform and a university course project.

➢ Tech Stack

Layer	Technology	Purpose
Frontend	Next.js 15 · React 19 · TypeScript · Tailwind CSS 3	SSG/SSR web application
UI Components	shadcn/ui · Radix UI · Lucide React	Accessible, composable component library
Backend	NestJS 11 · TypeScript	RESTful API server
Database	PostgreSQL 16 · Prisma ORM	Relational data storage
API	REST (JSON/HTTPS)	Frontend ↔ Backend communication
Payments	aamarPay	bKash · Nagad · Rocket · Cards

Layer	Technology	Purpose
Auth	JWT + Passport.js · bcrypt	Stateless authentication + password hashing
Email	Resend	Transactional email delivery
PDFs	pdf-lib	Watermarked e-books + programmatic certificates
Validation	class-validator · class-transformer · Zod	Server-side DTO + client-side form validation
Scheduling	@nestjs/schedule	Cron jobs (showcase verification)
Markdown	react-markdown · remark-gfm · rehype-raw	Chapter content rendering
Charts	Recharts	Dashboard data visualisation
Security	Helmet · @nestjs/throttler	Security headers + rate limiting
Hosting	Vercel + DigitalOcean Droplet	Frontend CDN + backend VPS
DNS/CDN	Cloudflare (free plan)	DNS · DDoS protection · CDN
SSL	Let's Encrypt (API) · Cloudflare (frontend)	HTTPS certificates
Process Manager	PM2	Production process management
CI/CD	GitHub Actions	Automated testing & deployment
Containerisation	Docker Compose	Local PostgreSQL development

➢ System Architecture

Architecture is not a single choice; it is a set of independent decisions across four dimensions. MultiHAT Academy makes a deliberate choice in each.

Dimension	Question	Decision	Rationale
Rendering Strategy	How and when is the HTML built?	**Hybrid Rendering**	Next.js 15 statically generates marketing and book-listing pages at build time **Static Site (SSG)** and server-renders user-specific pages per request **Server-Side Rendering (SSR)**: dashboard, payment confirmation, certificate viewer
Application Structure	How is the backend organised?	**Full-Stack Web App** (Headless/Decoupled pattern)	One NestJS monolith (12 modules, single PM2 process, one PostgreSQL 16 database) serves one frontend exclusively via a REST API contract; frontend and backend are independently deployed
Delivery Philosophy	Is content pre-built or fetched live?	**Jamstack**	The Next.js frontend is pre-built and served from Vercel's global CDN. All dynamic behaviour (purchases, quizzes, wallet, auth) is delegated to the NestJS REST API; no backend logic lives inside the frontend

Dimension	Question	Decision	Rationale
Infrastructure	Where does the code run?	**VPS (backend) + CDN (frontend)**	The API runs on a DigitalOcean Droplet (single VPS; no edge functions or serverless). The frontend static assets are served from Vercel's CDN, providing low-latency delivery globally for pre-rendered pages without edge computing for the API layer

☒ How the four decisions work together

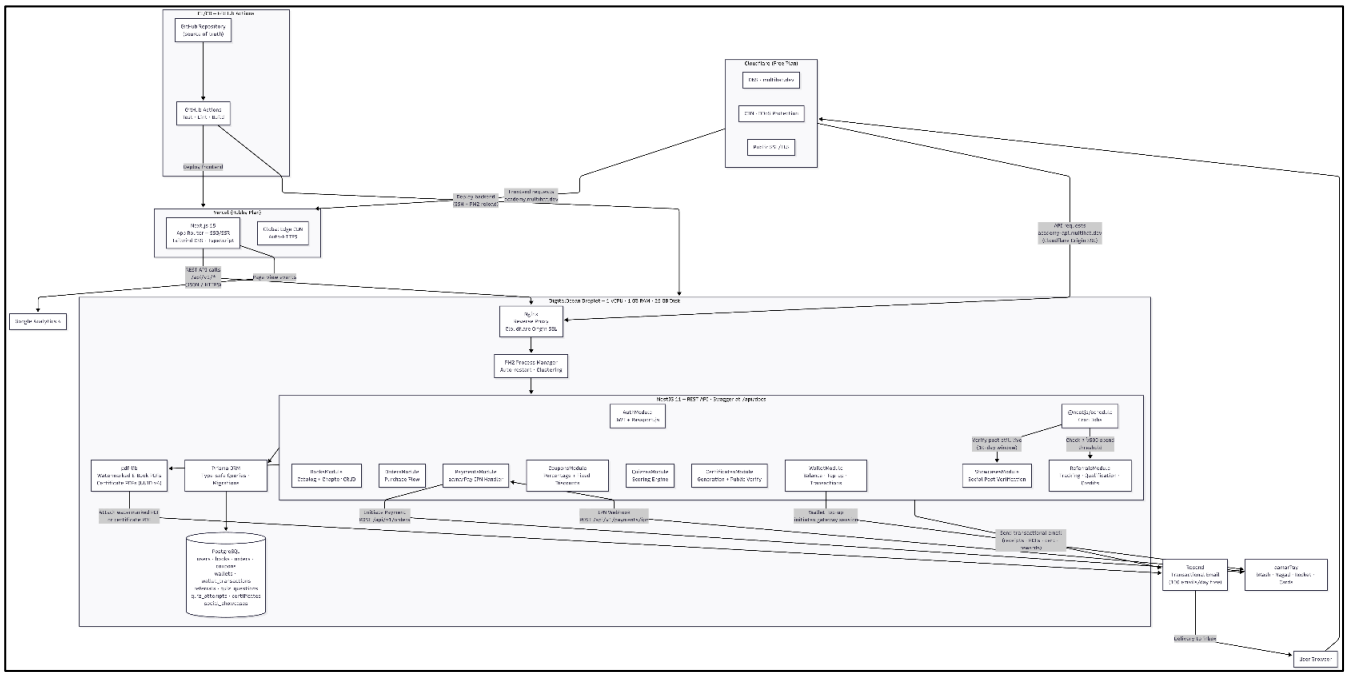
```
Browser
  │
  ├─→ Vercel CDN (edge PoP nearest to user)
  │     └── Next.js 15 — Hybrid Rendering
  │           ├── SSG: home · book listing · book detail   (pre-built — served instantly)
  │           └── SSR: dashboard · payment · certificate   (built per request — user-specific)
  │                        │
  │                        │  REST API (JSON/HTTPS) — Jamstack API layer
  │                        ▼
  └─→ DigitalOcean Droplet (single VPS)
        Nginx → PM2 → NestJS 11 Monolith (:5001)
          ├── 12 feature modules (auth · books · orders · payments · quizzes
          │                       certificates · wallet · referrals · showcases
          │                       users · coupons · email)
          └── Prisma ORM → PostgreSQL 16
```

Not used: Microservices (overkill for current scale; a monolith is easier to deploy, debug, and maintain on a 1 GB Droplet), Edge Computing for the API (not needed; the API is not globally distributed; only the frontend CDN provides geographic edge delivery).

Infrastructure

```
User → Cloudflare → Vercel (Next.js 15 — SSG/SSR)
                                  ↘
            DigitalOcean Droplet (1 vCPU · 1 GB · 25 GB)
              ├── Nginx (Reverse Proxy + Let's Encrypt SSL)
              ├── PM2 (fork mode · 800M restart) → NestJS 11 (:5001)
              └── Prisma ORM → PostgreSQL 16
                     ↓                  ↓
                 aamarPay            Resend
```

☒ System Architecture Diagram:



Live: [01-architecture-overview.png]

➢ Key Features

- **Premium E-books:** Watermarked PDFs with buyer's email tiled diagonally on every page (opacity 0.065, -45°)
- **Free Previews:** First 4 chapters of each book available without login (configurable via chapterMetadata.isFree)
- **Paid Web Chapters**: Premium gated web content purchasable via Wallet or payment gateway
- **Interactive Quizzes**: Multiple-choice assessments per book (70% pass threshold; hardcoded)
- **Verifiable Certificates**: Programmatic PDF certificates (pdf-lib, A4 landscape) with public verification at /verify/:certID
- **User Wallet**: Cash-in only wallet for platform purchases; funded via aamarPay top-ups, referral credits, and showcase rewards
- **Referral Program**: Earn 100 BDT/$0.80 per qualified referral (referred user must spend ≥ 500 BDT/$4); 3-step state machine: PENDING → QUALIFIED → CREDITED
- **Certification Showcase**: Earn Wallet credits by sharing certifications on LinkedIn (100 BDT), Twitter/X (100 BDT), Facebook (80 BDT), and Instagram (80 BDT); 10-day automated cron verification via HTTP HEAD
- **Local Payments**: aamarPay integration (bKash, Nagad, Rocket, cards) with IPN webhook + idempotency guards
- **Anti-Piracy**: Dynamic watermarks, UTM-tracked links, Google Alerts
- **Coupon System**: Percentage and fixed-amount discount codes with includes_pdf flag; codes auto-normalised to UPPERCASE
- **Dark Mode**: Full dark/light theme support (next-themes)
- **User Dashboard**: Purchase history, quiz scores, certificates, wallet balance, showcase submissions
- **Email Delivery**: Automatic watermarked PDF and purchase receipt delivery via Resend
- **Security:** Helmet headers, global rate limiting (100 req/60s), login throttle (10/min), DTO whitelist validation, CORS strict origin, Swagger disabled in production

---

➢ User Types & Features

☒ Types of User

MultiHAT Academy has three distinct user types, determined by authentication state and database role:

Type	Auth State	Role	How to become one
Guest	Not logged in	—	Visit the site without an account
Registered User	JWT authenticated	USER	POST /auth/register (default role)
Admin	JWT authenticated	ADMIN	Role set manually via PATCH /users/:id/role or DB seed

☒ Common Features (Available to All Users; No Login Required)

These endpoints work without a JWT token:

Feature	Endpoint	Notes
Browse published books	GET /books	Paginated list
View book detail	GET /books/:slug	No ownership flags returned
Read free chapters	GET /books/:slug/chapters/:index	Only chapters where isFree: true
View chapter media	GET /books/:slug/media/*	SSRF + path-traversal guarded
Validate a coupon	GET /coupons/verify/:code	Returns discount type and value
Verify a certificate	GET /certificates/verify/:certId	Returns {valid, holderName, courseTitle, issueDate}
Download a certificate	GET /certificates/:certId/pdf	Returns 404 if revoked
Register	POST /auth/register	Accepts optional referralCode
Log in	POST /auth/login	Throttled: 10 attempts/minute
Refresh session	POST /auth/refresh	Uses refresh token

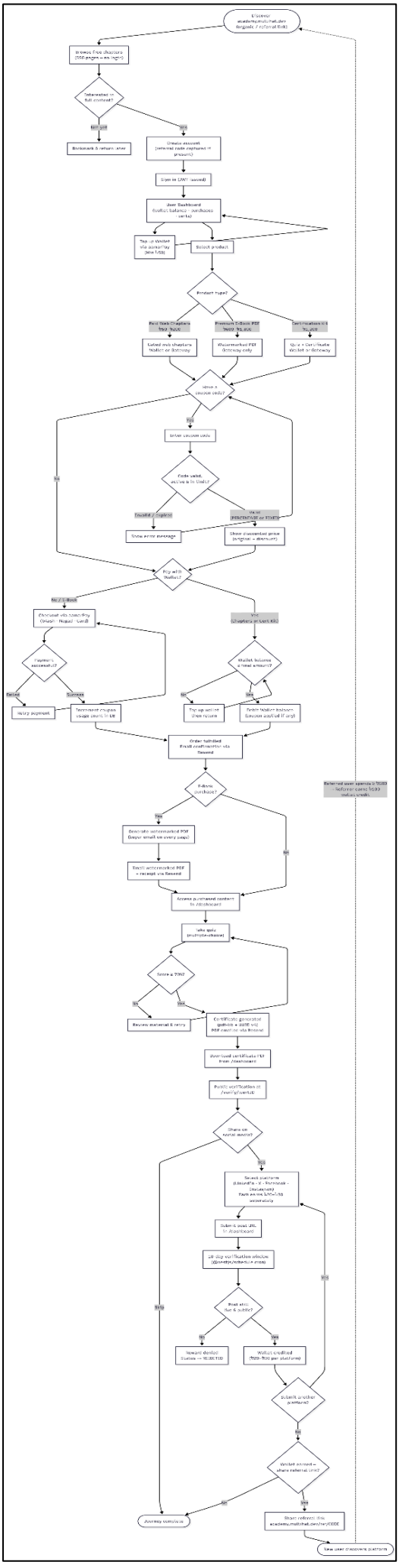
☒ Features by User Type

Registered User (Role.USER)

All common features above, plus:

Feature	Endpoint	Notes
View own profile	GET /users/me	{id, email, name, role, createdAt}
Update own name	PATCH /users/me	{name}
Book detail with ownership	GET /books/:slug	Returns isOwned, ownsPdf, hasPremiumPdf
Read paid chapters	GET /books/:slug/chapters/:index	Requires at least one PAID order for the book
Purchase a book	POST /orders	Gateway (aamarPay) or Wallet debit
View own orders	GET /orders/my	Full purchase history
Download watermarked e-book	GET /orders/:orderId/pdf	Ownership verified; PDF watermarked per buyer
Take a quiz	GET /quizzes/:bookSlug/questions	Requires PAID order; correct answers hidden
Submit quiz answers	POST /quizzes/:bookSlug/submit	Returns {score, total, outcome, certId?}; 70% = PASS → auto-issues certificate
View own certificates	GET /certificates/my	Ordered by date descending
Submit a showcase	POST /showcases/submit	3-layer guard: SSRF → ownership → no duplicate
View own showcase submissions	GET /showcases/my	Includes verification status
Wallet balance	GET /wallet/balance	{balanceBdt, lifetimeEarned, lifetimeSpent}
Top up wallet	POST /wallet/topup	Initiates aamarPay payment → redirects to gateway
Confirm wallet top-up	POST /wallet/topup/confirm	Polls aamarPay search API as a fallback
Wallet transaction history	GET /wallet/transactions	Paginated (default 20 · max 100)
Get referral link	GET /referrals/code	{referralCode, referralLink}
Referral statistics	GET /referrals/stats	{total, pending, qualified, credited, totalEarned}

P.T.O



☒ **User Journey Diagram:**

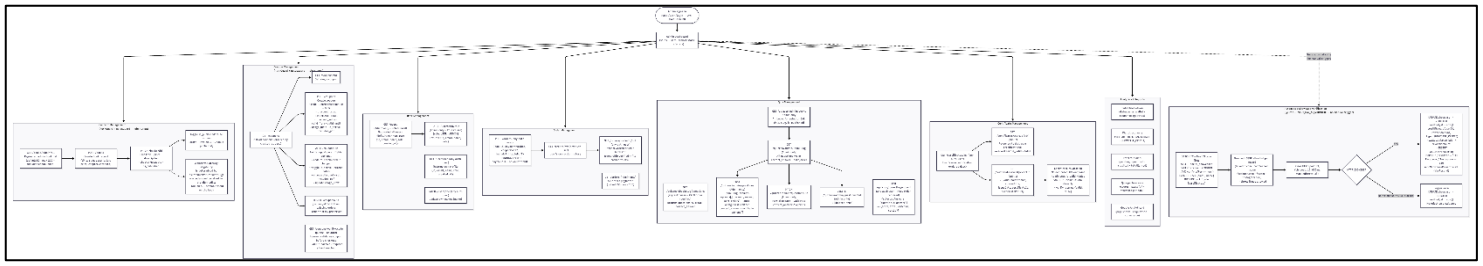
Live: [03-user-journey.png]

☒ Admin (Role.ADMIN)

All registered user features above, plus exclusive admin-only access:

Feature	Endpoint	Notes
List all books (incl. unpublished)	GET /books/admin/all	Paginated (default 50 · max 200)
Create a book	POST /books	{title, slug, description, price, chapterMetadata}
Update a book	PATCH /books/:id	Toggle is_published, update price, metadata
List all coupons	GET /coupons	Ordered by created_at desc
Get a single coupon	GET /coupons/:id	
Create a coupon	POST /coupons	Code auto-normalised to UPPERCASE; PERCENTAGE or FIXED
Update a coupon	PATCH /coupons/:id	All fields patchable; code re-normalised if changed
Delete a coupon	DELETE /coupons/:id	coupon_id set to NULL on existing orders (history preserved)
View all orders	GET /orders	Paginated (default 50 · max 100)
List all users	GET /users	Paginated (default 50 · max 100)
Change a user's role	PATCH /users/:id/role	{role: "USER" | "ADMIN"}
Book selector for quiz admin	GET /quizzes/admin/books	{id, title, slug, isPublished}
List questions with answers	GET /quizzes/admin/:bookSlug	Includes correct_answer + sort_order
Create a quiz question	POST /quizzes/admin/questions	correct_answer must be in options[]
Update a quiz question	PATCH /quizzes/admin/questions/:id	Validates correct_answer ∈ options
Delete a quiz question	DELETE /quizzes/admin/questions/:id	Returns {deleted: true}
Revoke a certificate	DB / Prisma Studio directly	Set is_valid = false; no dedicated API endpoint

☒ **Admin Workflow Diagram:**



Live: [06-admin-workflow.png]

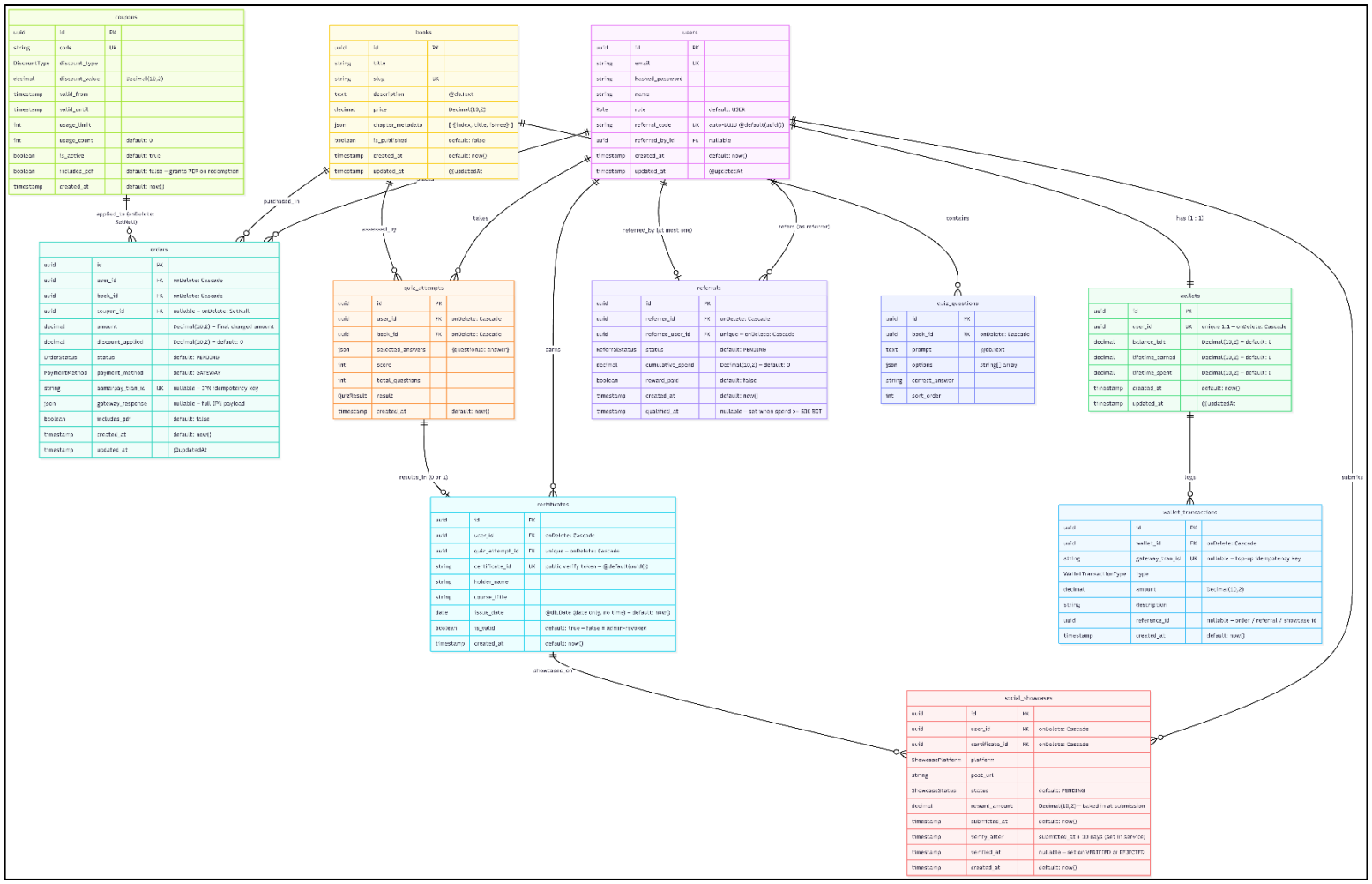
➢ **Data Model**

The PostgreSQL database is managed via Prisma ORM. The schema is built around 11 core entities, designed with strict referential integrity (Cascade / SetNull) and financial safety (Decimal types, raw SQL atomic decrements).

☒ **Enums**

Enum	Values
Role	USER · ADMIN
OrderStatus	PENDING · PAID · FAILED · REFUNDED
PaymentMethod	GATEWAY · WALLET
DiscountType	PERCENTAGE · FIXED
QuizResult	PASS · FAIL
ReferralStatus	PENDING · QUALIFIED · CREDITED
ShowcasePlatform	LINKEDIN · TWITTER · FACEBOOK · INSTAGRAM
ShowcaseStatus	PENDING · VERIFIED · REJECTED
WalletTransactionType	TOPUP · PURCHASE · REFERRAL_CREDIT · SHOWCASE_CREDIT

☒ Entity Relationship Diagram



Live: [04-data-model.png]

☒ Constraints & Cascade Rules

Table	Column / Index	Type	Behaviour
orders	coupon_id FK	onDelete: SetNull	Deleting a coupon nulls coupon_id; order history preserved
orders	user_id FK	onDelete: Cascade	Deleting a user cascades to their orders
orders	book_id FK	onDelete: Cascade	Deleting a book cascades to its orders
orders	aamarpay_tran_id	@unique	Prevents duplicate IPN processing
quiz_questions	book_id FK	onDelete: Cascade	Deleting a book cascades to its quiz questions
quiz_attempts	user_id FK	onDelete: Cascade	Deleting a user cascades to their attempts
quiz_attempts	book_id FK	onDelete: Cascade	Deleting a book cascades to its attempts
certificates	quiz_attempt_id	@unique	One certificate per quiz attempt maximum

Table	Column / Index	Type	Behaviour
certificates	quiz_attempt_id FK	onDelete: Cascade	Deleting the attempt cascades to the certificate
certificates	user_id FK	onDelete: Cascade	Deleting a user cascades to their certificates
coupons	*(no FK)*	—	No cascades; referenced via orders.coupon_id (SetNull)
wallets	user_id	@unique	Enforces 1:1 relationship with users
wallets	user_id FK	onDelete: Cascade	Deleting a user cascades to their wallet
wallet_transactions	wallet_id FK	onDelete: Cascade	Deleting a wallet cascades to its transactions
wallet_transactions	gateway_tran_id	@unique nullable	Idempotency guard for top-up IPN webhooks (P2002 silently ignored)
referrals	referred_user_id	@unique	A user can be referred at most once
referrals	referrer_id FK	onDelete: Cascade	Deleting referrer cascades to referral records
referrals	referred_user_id FK	onDelete: Cascade	Deleting referred user cascades to referral records
social_showcases	(user_id, certificate_id, platform)	@@unique	One reward per platform per certificate per user
social_showcases	user_id FK	onDelete: Cascade	Deleting a user cascades to their showcase submissions
social_showcases	certificate_id FK	onDelete: Cascade	Deleting a certificate cascades to its showcases

☒ Key Design Notes

Concern	Detail
All PKs	@id @default(uuid()) @db.Uuid; PostgreSQL native UUID
Financial fields	All monetary values use Decimal(10,2); no floating point
certificate_id vs id	certificates.id = internal DB PK · certificates.certificate_id = public verify token (also a UUID, auto-generated separately)
referral_code	Every user gets a unique referral code at registration; @default(uuid())
issue_date	@db.Date; stored as a pure date with no time component
social_showcases.reward_amount	Baked in at INSERT time from PLATFORM_REWARDS constant; not recomputed by cron
social_showcases.verified_at	Set on BOTH VERIFIED and REJECTED terminal states
coupons has no updated_at	Intentional; no @updatedAt in schema

Concern	Detail
quiz_questions has no timestamps	Intentional; no created_at or updated_at in schema
Wallet debit atomicity	Uses raw SQL UPDATE wallets SET balance_bdt = balance_bdt - :amt WHERE balance_bdt >= :amt; eliminates TOCTOU race

---

➢ REST API Endpoints

All endpoints are prefixed with /api/v1. Global rate limit: 100 requests per 60 seconds per IP.

Group	Method	Endpoint	Auth	Notes
Auth	POST	/auth/register	No	Accepts optional referralCode
	POST	/auth/login	No	Throttled: 10 attempts/minute
	POST	/auth/refresh	Refresh token	
Books	GET	/books	No	Published only · paginated (default 20 · max 100)
	GET	/books/admin/all	Admin	All books incl. unpublished (default 50 · max 200)
	GET	/books/:slug	Optional	Returns isOwned, ownsPdf, hasPremiumPdf, requiresGatewayPayment
	GET	/books/:slug/chapters/:index	Optional	Free chapters: no auth · Paid: 403 without order
	GET	/books/:slug/media/*	No	Image serving with SSRF + path-traversal guard
	POST	/books	Admin	Create book
	PATCH	/books/:id	Admin	Update book fields
Coupons	GET	/coupons/verify/:code	No	Learner validates coupon before checkout
	GET	/coupons	Admin	List all coupons
	GET	/coupons/:id	Admin	Single coupon
	POST	/coupons	Admin	Create; code UPPER-normalised
	PATCH	/coupons/:id	Admin	Partial update
	DELETE	/coupons/:id	Admin	Soft cascade; SetNull on orders
Orders	POST	/orders	Yes	Create order (gateway or wallet)
	GET	/orders/my	Yes	Own order history

Group	Method	Endpoint	Auth	Notes
	GET	/orders/:orderId/pdf	Yes	Stream watermarked e-book PDF
	GET	/orders	Admin	All orders paginated
Payments	POST	/payments/ipn	Webhook	aamarPay IPN; idempotent, signature-verified
	GET/POST	/payments/success	Redirect	Gateway return; verifies via search API
	GET/POST	/payments/fail	Redirect	Marks order FAILED, releases coupon
	GET/POST	/payments/cancel	Redirect	Same as fail with reason "cancelled"
Quizzes	GET	/quizzes/:bookSlug/questions	Yes	Without correct answers (requires PAID order)
	POST	/quizzes/:bookSlug/submit	Yes	Returns score + optional certificate
	GET	/quizzes/admin/books	Admin	Book selector list
	GET	/quizzes/admin/:bookSlug	Admin	Questions with correct answers
	POST	/quizzes/admin/questions	Admin	Create question
	PATCH	/quizzes/admin/questions/:id	Admin	Update question
	DELETE	/quizzes/admin/questions/:id	Admin	Delete question
Certificates	GET	/certificates/my	Yes	Own certificates
	GET	/certificates/:certId/pdf	No	Re-generate & stream certificate PDF
	GET	/certificates/verify/:certId	No	Public verification
Users	GET	/users/me	Yes	Own profile
	PATCH	/users/me	Yes	Update own name
	GET	/users	Admin	All users paginated (default 50 · max 100)
	PATCH	/users/:id/role	Admin	Role promotion/demotion
Wallet	GET	/wallet/balance	Yes	{balanceBdt, lifetimeEarned, lifetimeSpent}

Group	Method	Endpoint	Auth	Notes
	POST	/wallet/topup	Yes	Initiate top-up → aamarPay URL
	POST	/wallet/topup/confirm	Yes	Confirm via aamarPay search API
	GET	/wallet/transactions	Yes	Paginated history (default 20 · max 100)
Referrals	GET	/referrals/code	Yes	{referralCode, referralLink}
	GET	/referrals/stats	Yes	{total, pending, qualified, credited, totalEarned}
Showcases	POST	/showcases/submit	Yes	3 guards: SSRF → ownership → duplicate
	GET	/showcases/my	Yes	Own submissions with status
Health	GET	/healthz	No	{status: "ok", uptime}

➢ CI/CD Pipeline

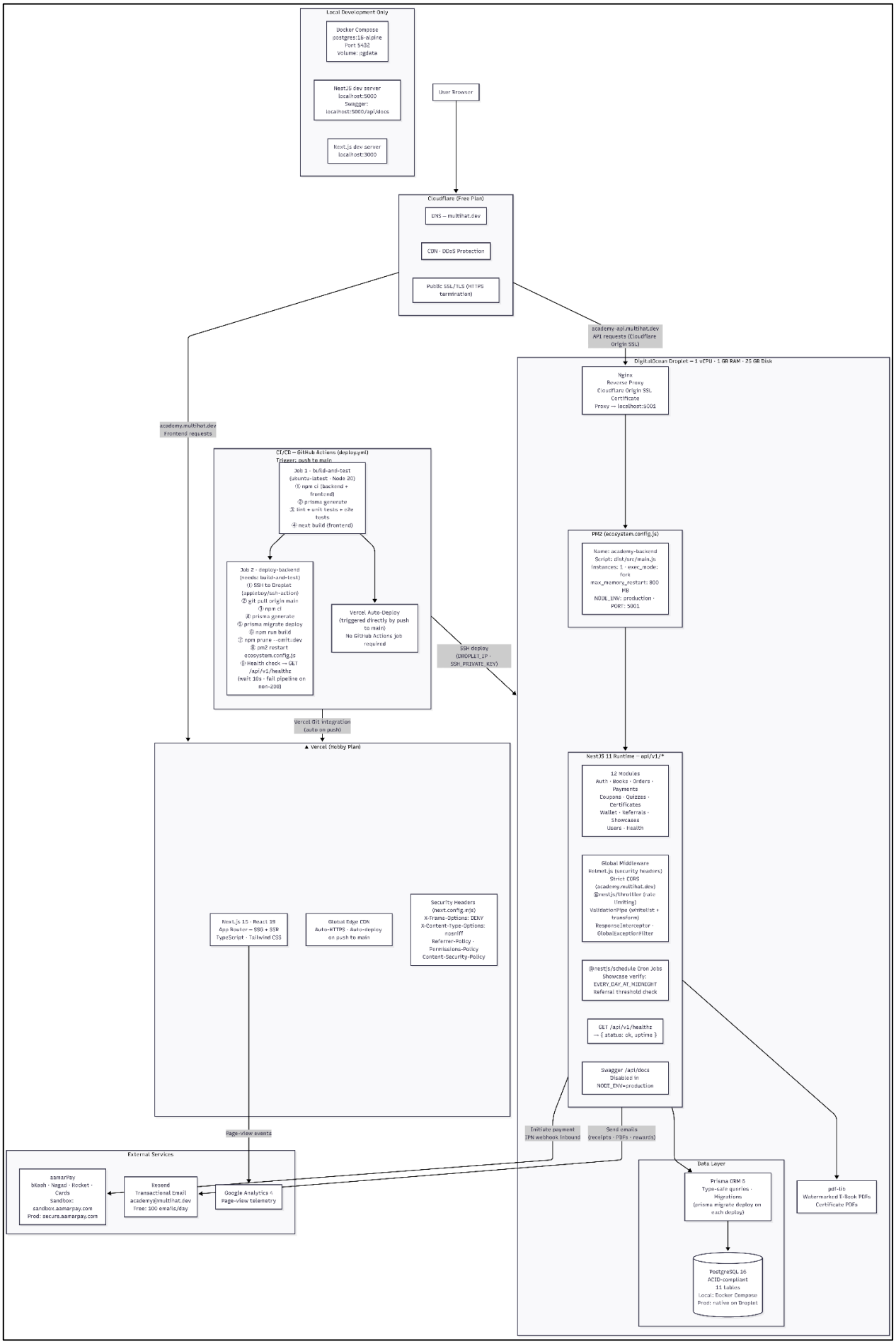
Triggered on push to main via GitHub Actions:

Job	Steps
build-and-test	npm ci → prisma generate → lint → test → test:e2e (backend) · npm ci → build (frontend)
deploy-backend	SSH to Droplet → git pull → npm ci → prisma generate → prisma migrate deploy → npm run build → npm prune --omit=dev → pm2 restart
health-check	Wait 10s → curl GET /healthz → fail pipeline if non-200

P.T.O

☒ Diagram:

Live: [05-deployment-topology.png]



➢ **Budget**

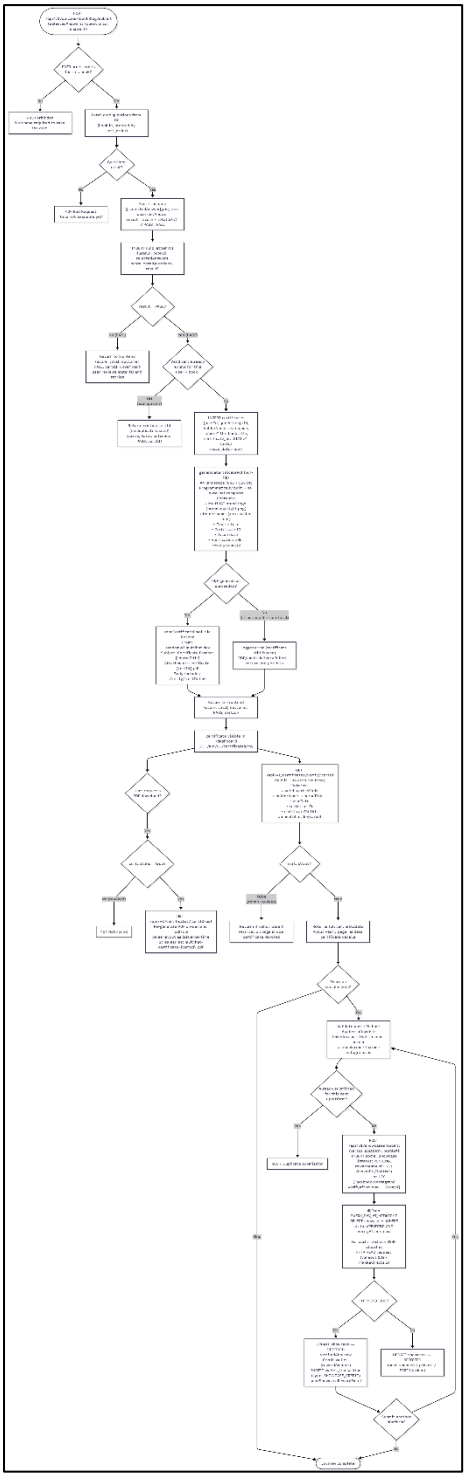
Item	Cost
Domain (academy.multihat.dev)	$0; free subdomain
Cloudflare DNS & CDN	$0; free plan
DigitalOcean Droplet	$0 additional; existing
Vercel Hosting	Free (Hobby)
Resend Email	Free (100/day)
Let's Encrypt SSL	$0; free
aamarPay Fees	~2% per transaction

Total additional fixed cost: $0/year

➢ **Certification Design:**



☒ Certificate Issuance Flow Diagram:



Live: [07-certificate-issuance-flow.png]

☒ All 8+7 diagrams are available on: [Diagrams]; For high-quality, please download the diagrams.

☒ GitHub Repository: [SagarBiswas-MultiHAT/Multihat-Academy]

---------- X ----------